Classification multimodale de produits e-commerce : prédiction des codes produits Rakuten

Auteurs : Noel Ching, Ediz Ozkohen

Formation : Formation Data Scientist

Établissement : Liora (ex. DataScientest)

Tuteur : Antoine Fradin

Date : 20 Mars 2026

Sommaire

Sommaire 2
1. Introduction 4
2. Description du dataset 4
Particularités du dataset 5
3. Exploration des données (EDA) 5
3.1 Structure générale et qualité des données 5
Valeurs manquantes 5
3.2 Analyse de la variable cible 6
Déséquilibre des classes 6
3.3 Analyse du déséquilibre et simulation SMOTE 7
3.4 Analyse des données textuelles 8
Présence des descriptions 8
Caractéristiques textuelles 9
Analyse par catégorie 9
3.5 Analyse sémantique du texteInterprétation : 10
3.6 Analyse exploratoire des images 11
Disponibilité et qualité 11
Caractéristiques des images 11
Résultats principaux 12
3.7 Relation entre texte et image 12
3.9 Synthèse des insights 14
Conclusion de l'EDA 14
4. Préprocessing des données 15
4.1 Nettoyage et normalisation du texte 15
Étapes de nettoyage 15
4.2 Vectorisation TF-IDF et réduction de dimension 16
Paramétrage 16
Réduction de dimension (SVD) 16
4.3 Embeddings sémantiques avec CamemBERT 17
Méthodologie 17
4.4 Extraction des caractéristiques images 17
4.4.1 Features simples (metadata) 17
4.4.2 Embeddings visuels avec EfficientNet 18
Méthodologie 18
Réduction de dimension (SVD) 18
4.5 Fusion multimodale des features 19
Composition du vecteur final 19
4.6 Construction du dataset final 19
Vérifications 19
4.7 Optimisation et gestion mémoire 20
4.8 Synthèse du preprocessing 20

5. Modèles et méthodologie expérimentale.....21
5.1 Choix des modèles et hypothèses.....21
5.2 Description détaillée des modèles.....21
5.2.1. Régression Logistique.....21
5.2.2. LightGBM.....21
5.2.3. CatBoost.....22
5.2.4. Multi-Layer Perceptron (MLP).....22
5.2.5. Stacking.....23
5.3 Synthèse comparative.....23
6. Interprétabilité.....24
7. Conclusion scientifique.....24
1. Importance des embeddings.....24
2. Supériorité des modèles non linéaires.....24
3. Limite du stacking.....24
4. Impact du déséquilibre des classes.....24
8. Conclusion métier.....25
Contexte.....25
Apports business.....25
Modèle retenu.....25
Cas d'usage SaaS.....25
9. Perspectives futures.....26
9.1 Améliorations possibles.....26

1. Introduction

Le développement des plateformes e-commerce a profondément transformé les modes de consommation, avec une croissance exponentielle du nombre de produits disponibles en ligne. Dans ce contexte, la structuration et la classification des produits constituent un enjeu central pour améliorer l'expérience utilisateur, notamment en matière de recherche, de recommandation et de navigation.

Rakuten, acteur majeur du e-commerce mondial, fait face à ces problématiques à grande échelle. La plateforme repose sur un modèle de marketplace, où des vendeurs professionnels et particuliers publient des annonces produits, souvent avec des données hétérogènes et bruitées (titres imprécis, descriptions incomplètes, images de qualité variable).

Dans ce cadre, le présent projet s'inscrit dans un challenge proposé par le département de recherche de Rakuten, visant à développer un modèle de classification automatique des produits. L'objectif est de prédire le **code type produit (prdtypecode)** à partir de données multimodales, combinant texte (titre et description) et image.

Ce problème présente plusieurs difficultés majeures :

- **Nature multimodale des données** (texte + image)
- **Qualité variable des données** (bruit, valeurs manquantes)
- **Grand nombre de classes**
- **Forte déséquilibration du dataset**

L'objectif de ce projet est donc de concevoir un pipeline de Machine Learning capable d'exploiter efficacement ces différentes sources d'information afin d'optimiser la performance de classification.

La métrique retenue pour évaluer les performances est le **Weighted F1-score**, particulièrement adaptée aux problèmes de classification multi-classes déséquilibrés.

2. Description du dataset

Le dataset fourni dans le cadre du challenge contient environ **99 000 produits**, répartis entre un jeu d'entraînement et un jeu de test :

- **Train** : 84 916 observations
- **Test** : 13 812 observations

Chaque produit est décrit à travers plusieurs variables :

- **designation** : titre du produit (texte court)
- **description** : description détaillée (souvent absente)
- **productid** : identifiant unique du produit
- **imageid** : identifiant de l'image associée
- **prdtypecode** : catégorie cible (uniquement pour le train)

Les images sont fournies séparément et doivent être associées aux produits via les identifiants.

Particularités du dataset

Plusieurs caractéristiques rendent ce dataset complexe :

- **Données textuelles bruitées** :
  - fautes de frappe
  - encodage incorrect
  - langues multiples

- **Descriptions manquantes** :
  - une proportion importante de valeurs nulles

- **Images hétérogènes** :
  - qualité variable
  - angles et fonds différents

- **Distribution déséquilibrée des classes** :
  - certaines catégories sont largement sur-représentées

Ces éléments imposent des choix méthodologiques spécifiques, notamment en preprocessing et en modélisation.

3. Exploration des données (EDA)

L'analyse exploratoire des données a été réalisée afin de comprendre la structure du dataset, identifier ses spécificités et orienter les choix de modélisation. Cette étape est particulièrement importante dans le cadre de ce projet, compte tenu de la nature **multimodale et bruitée** des données.

3.1 Structure générale et qualité des données

Le dataset d'entraînement est constitué d'environ **85 000 observations**, chacune correspondant à un produit.

Une première analyse met en évidence :

- la présence de variables textuelles (*designation*, *description*)
- des identifiants techniques (*productid*, *imageid*)
- une variable cible (*prdtypecode*)

Valeurs manquantes

L'analyse des valeurs manquantes révèle un point critique :

- une proportion significative des **descriptions est absente (environ 35%)**

Cela implique que :

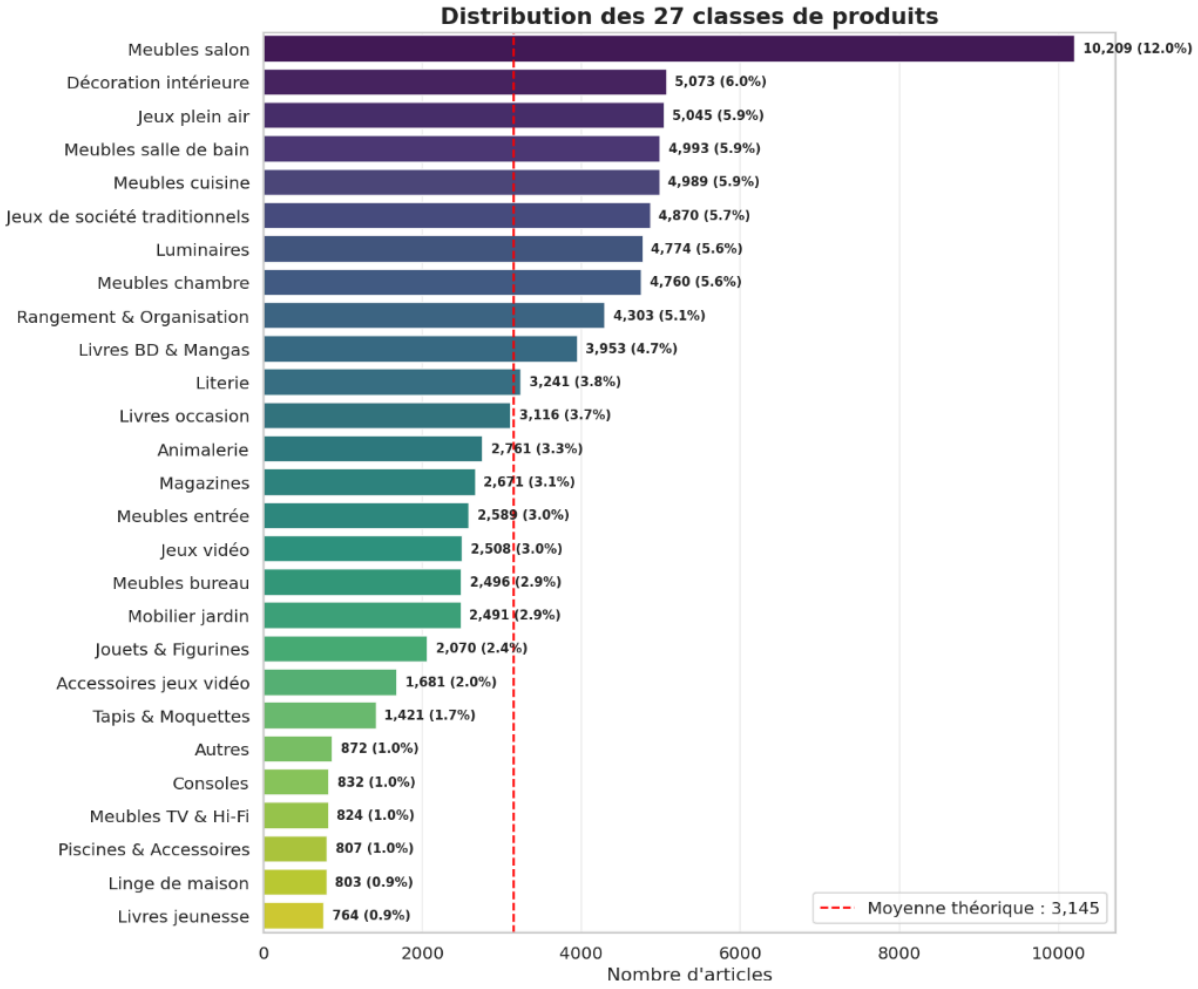
- la variable *designation* (titre produit) joue un rôle central
- les modèles doivent être robustes à l'absence de description

3.2 Analyse de la variable cible

Le dataset comporte **27 classes de produits**, correspondant à différentes catégories métier (livres, meubles, jeux, etc.).

Déséquilibre des classes

La distribution des classes est **fortement déséquilibrée** :



Ce déséquilibre a plusieurs implications :

- nécessité d'utiliser une métrique adaptée (**Weighted F1-score**)
- besoin potentiel de techniques de rééquilibrage (ex : **SMOTE** ou pondération des classes)

Interprétation :

- certaines classes dominent largement (ex :meubles, jeux)
- d'autres sont très faiblement représentées
- la distribution est loin d'être uniforme
- la ligne de moyenne théorique met en évidence cet écart

Impact :

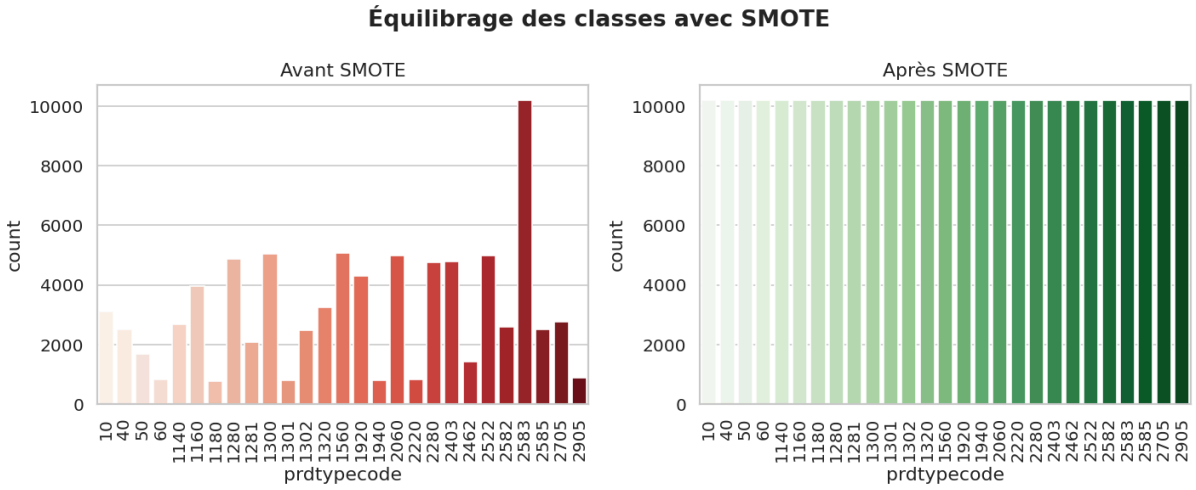
- utilisation du **Weighted F1-score**
- nécessité de techniques de rééquilibrage (pondération ou sur-échantillonnage)

3.3 Analyse du déséquilibre et simulation SMOTE

Afin de mieux comprendre l'impact du déséquilibre des classes, une simulation a été réalisée à l'aide de la méthode **SMOTE (Synthetic Minority Over-sampling Technique)**.

Cette méthode permet de générer artificiellement de nouvelles observations pour les classes minoritaires, à partir de leurs voisins dans l'espace des features.

Dans cette analyse, SMOTE est appliqué sur un sous-ensemble de variables textuelles simples (longueur, nombre de mots, etc.) afin d'illustrer son effet sur la distribution des classes.



Interprétation :

- avant SMOTE : distribution très déséquilibrée
- après SMOTE : répartition beaucoup plus homogène entre classes

Impact :

- confirme que le déséquilibre est un problème majeur
- valide l'intérêt de techniques de rééquilibrage

Il est important de noter que :

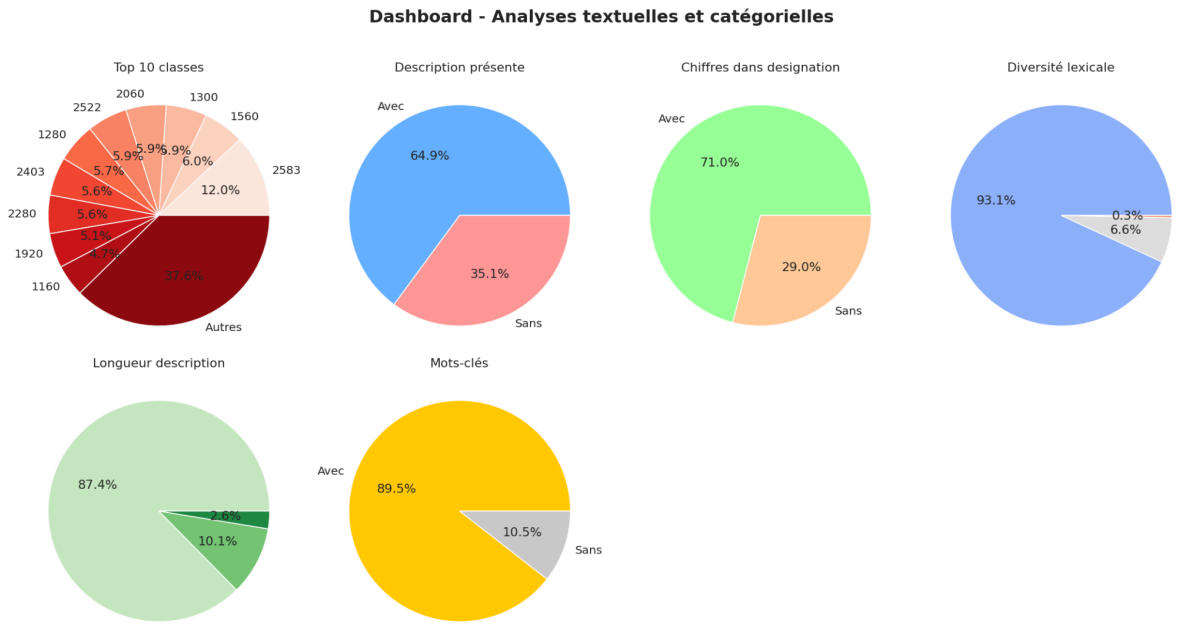
- cette transformation est réalisée **uniquement à des fins exploratoires**
- elle ne sera pas nécessairement appliquée telle quelle au modèle final

3.4 Analyse des données textuelles

Les variables textuelles constituent une source d'information majeure pour la classification.

Présence des descriptions

Toutes les annonces ne contiennent pas de description.



Interprétation :

- une proportion importante d'annonces ne contient pas de description
- la qualité de l'information est donc hétérogène

Impact :

- le modèle devra s'appuyer fortement sur *designation*
- la description est une feature utile mais non fiable seule

Caractéristiques textuelles

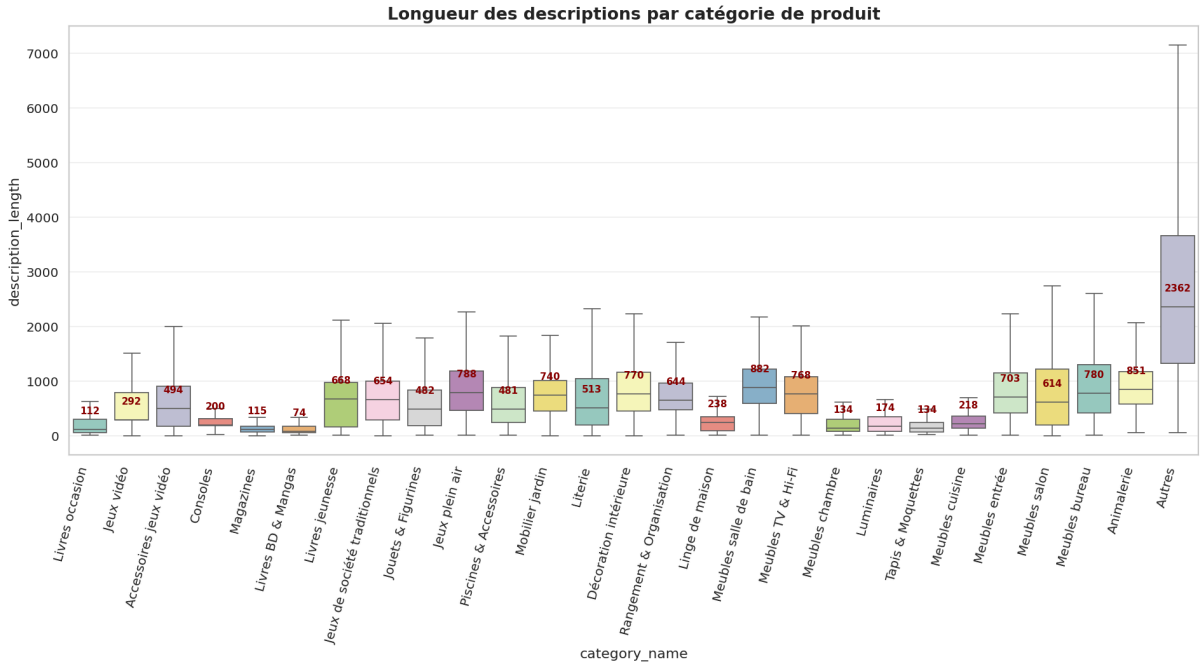
Plusieurs features ont été construites pour analyser le texte :

- longueur des titres et descriptions
- nombre de mots
- présence de chiffres
- diversité lexicale

Ces variables permettent de capturer des différences structurelles entre produits.

Analyse par catégorie

Une analyse des longueurs de description par catégorie met en évidence des différences importantes entre classes.



Interprétation :

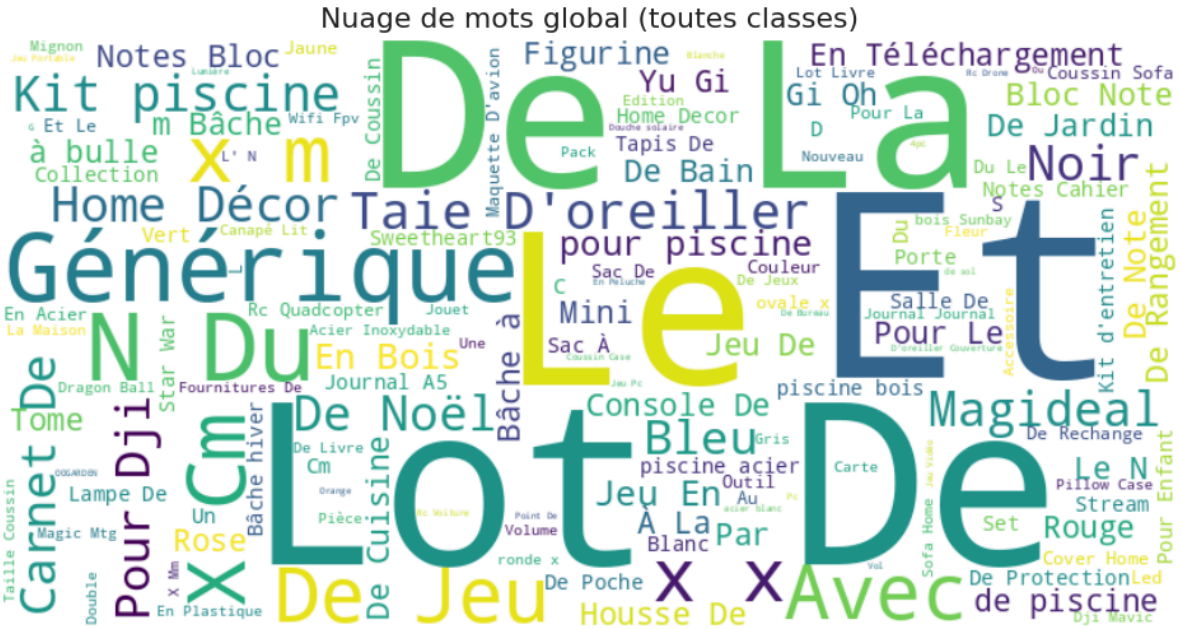
- certaines catégories (ex : mobilier, décoration) ont des descriptions longues
- d'autres reposent sur des textes courts
- forte de variabilité intra-classe

Un test **ANOVA** a confirmé que ces différences sont **statistiquement significatives**.

Impact :

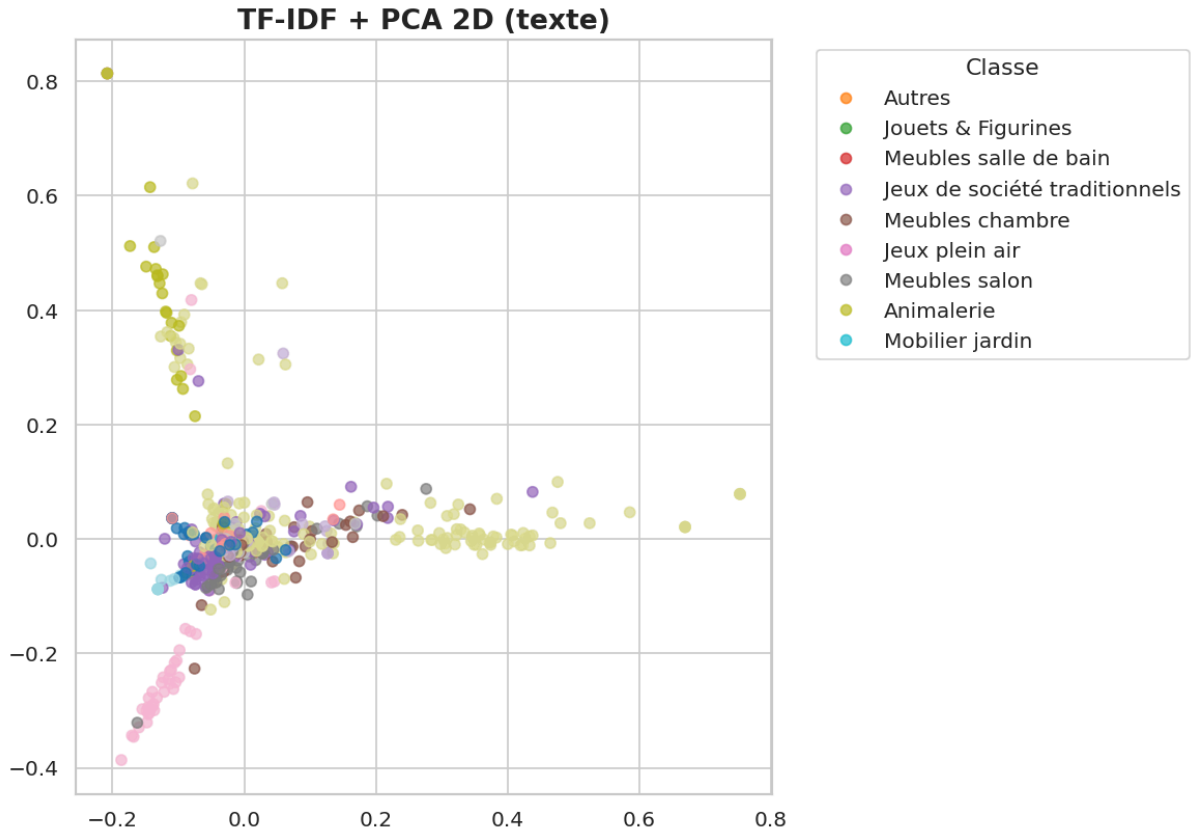
- la longueur de description est une feature discriminante
- le texte contient une information structurante pour la classification

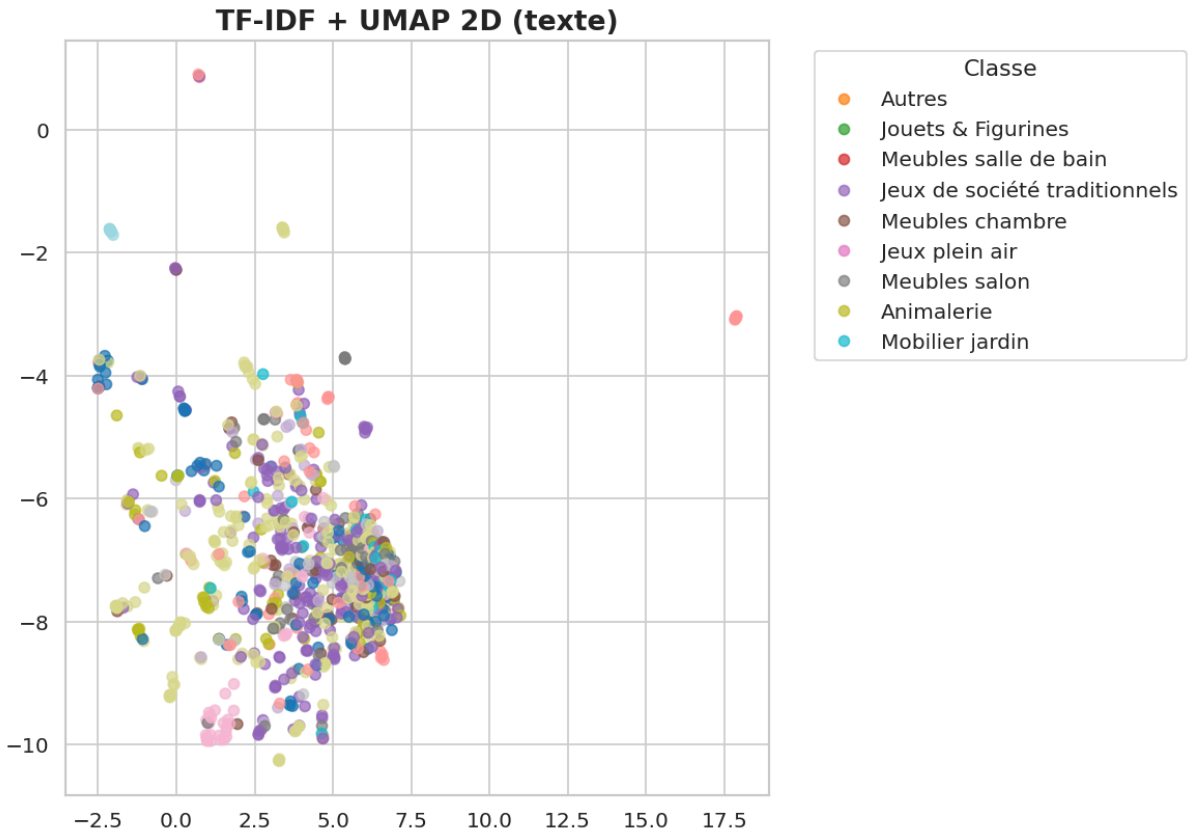
3.5 Analyse sémantique du texte



Interprétation :

- identification de mots-clés fréquents (ex : “lot”, “pack”, “neuf”)





Interprétation :

- certaines classes sont bien séparées
- d'autres se chevauchent

Impact :

- le texte est discriminant mais pas suffisant seul

3.6 Analyse exploratoire des images

Les images constituent la seconde modalité du dataset.

Disponibilité et qualité

Une vérification des fichiers montre que :

- la majorité des images est disponible
- certaines images peuvent être absentes ou corrompues

Caractéristiques des images

Plusieurs propriétés ont été extraites :

- largeur et hauteur (résolution)
- taille du fichier (en Ko)

Résultats principaux

L'analyse révèle que :

- les images présentent une **grande variabilité de dimensions**
- la taille des fichiers varie fortement selon les catégories
- certaines catégories (ex : mobilier) ont tendance à avoir des images plus lourdes et de meilleure qualité

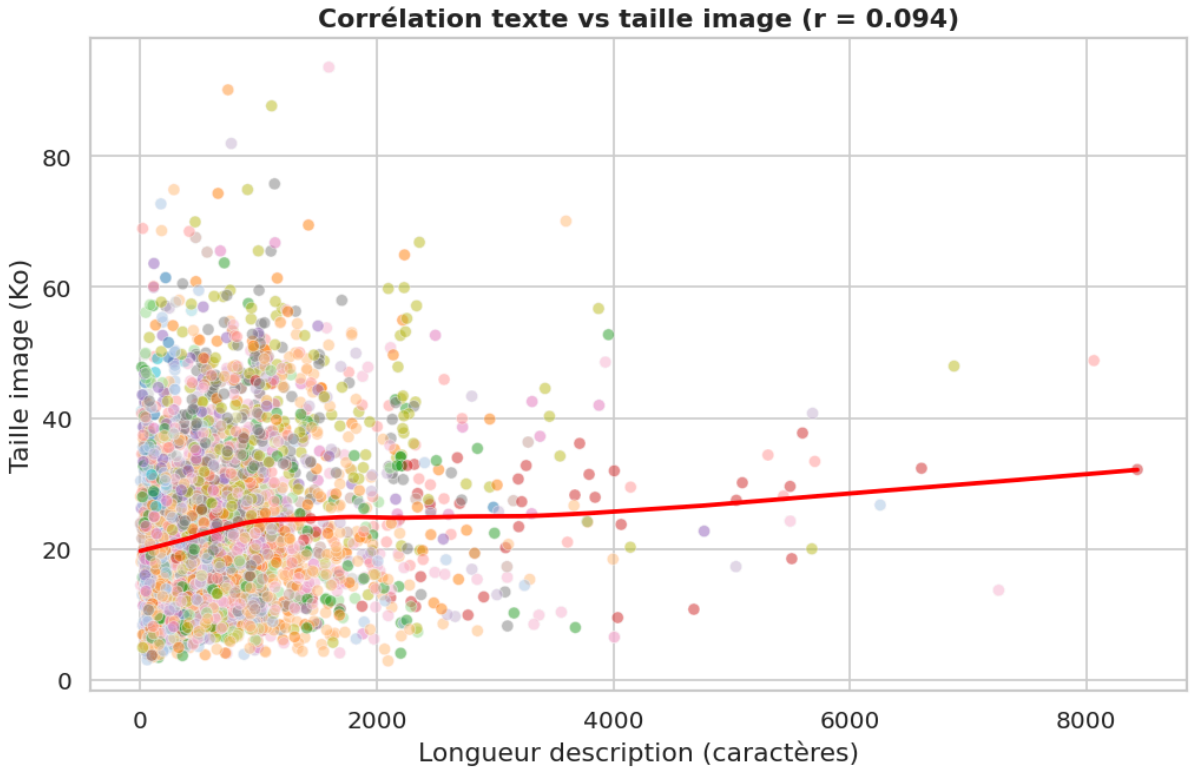
Ces différences suggèrent que les caractéristiques visuelles peuvent être discriminantes.

3.7 Relation entre texte et image

Une analyse de corrélation entre :

- la longueur des descriptions
- la taille des images

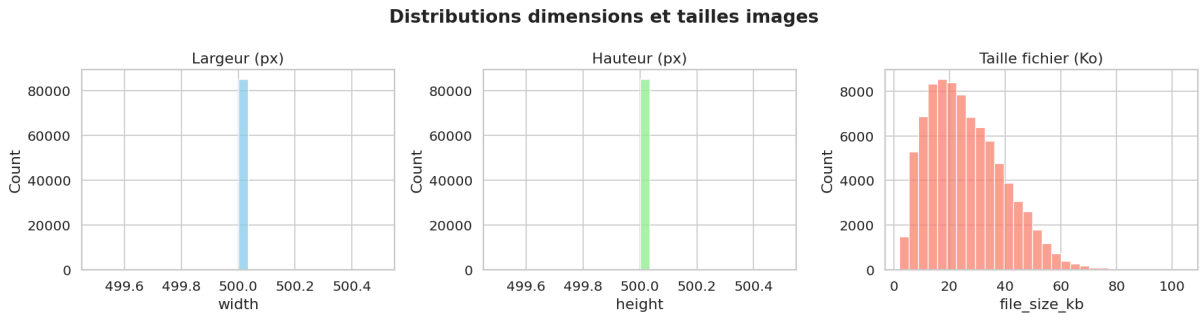
met en évidence une **corrélation positive modérée**.



Cela signifie que :

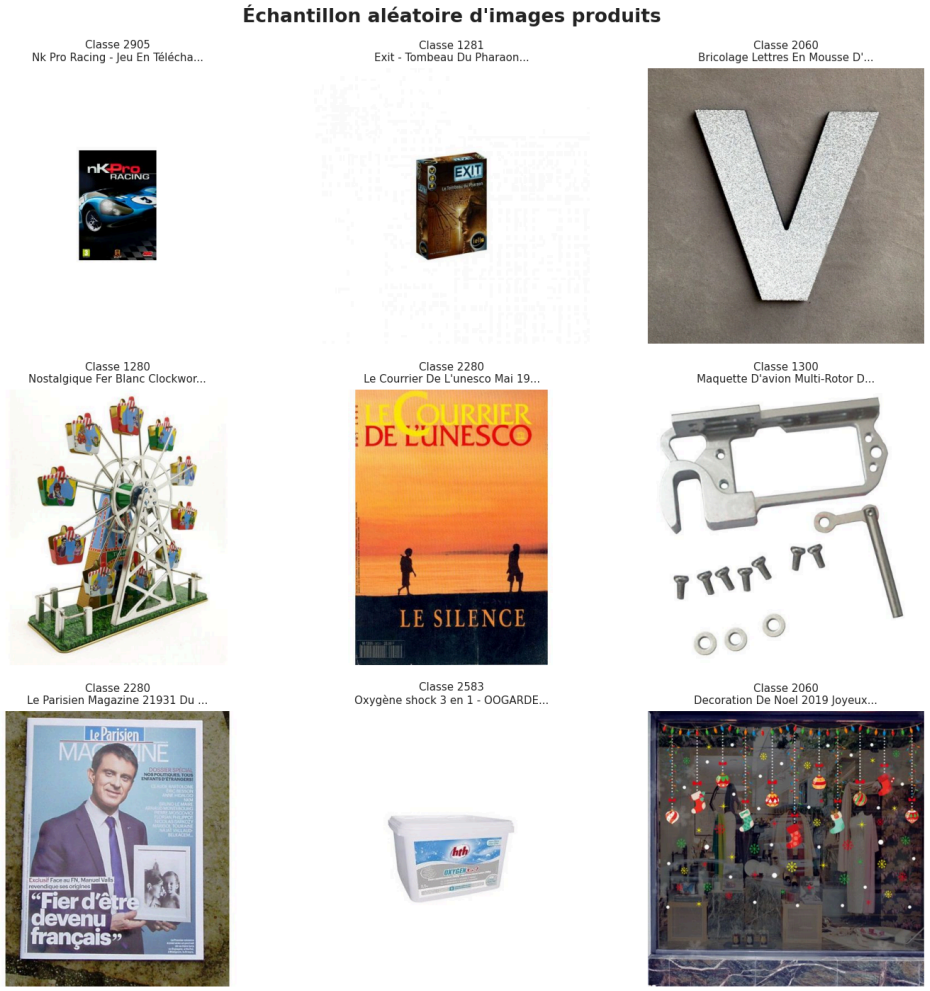
- les produits avec des descriptions longues ont tendance à être associés à des images plus riches ou plus détaillées
- les deux modalités sont **complémentaires**

Ce résultat justifie pleinement l'utilisation d'une approche multimodale.



Interprétation de la distribution des dimensions et des tailles images :

- forte variabilité des images



Interprétation de l’échantillon aléatoire d’images produits :

- diversité visuelle importante
- présence de bruit

3.9 Synthèse des insights

L'analyse exploratoire permet de dégager plusieurs enseignements clés :

- **Fort déséquilibre des classes**, nécessitant une stratégie adaptée
- **Descriptions manquantes (~35%)**, renforçant l'importance des titres
- **Variabilité importante des données textuelles et visuelles**
- **Différences significatives entre classes**, exploitables par les modèles
- **Complémentarité entre texte et image**
- **Présence de signaux discriminants** dans les deux modalités

Conclusion de l'EDA

L'EDA confirme que le problème est **complexe mais riche en information**.

Les résultats orientent clairement vers :

- une approche **multimodale**
- l'utilisation de **features textuelles avancées**
- l'intégration de **représentations visuelles**
- la prise en compte du **déséquilibre des classes**

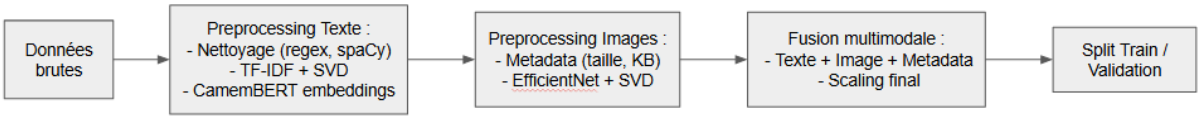
Ces éléments serviront de base pour la phase de preprocessing et de modélisation.

4. Préprocessing des données

Cette étape vise à transformer les données brutes en un ensemble de variables exploitables par les modèles de machine learning.
Compte tenu de la nature multimodale du dataset, le preprocessing est structuré autour de trois axes :

- traitement du texte
- extraction des caractéristiques images
- fusion des différentes sources d'information

Voici notre schéma de preprocessing que nous allons détailler par la suite :



4.1 Nettoyage et normalisation du texte

Les variables textuelles (*designation* et *description*) ont fait l'objet d'un nettoyage avancé afin de réduire le bruit et homogénéiser les données.

Étapes de nettoyage

Le pipeline de preprocessing textuel comprend :

- conversion en minuscules
- normalisation Unicode (suppression des accents)
- suppression des balises HTML et URLs
- suppression des caractères non alphanumériques
- suppression des unités de mesure (cm, kg, etc.)
- suppression des valeurs numériques
- suppression de mots anglais fréquents
- lemmatisation via **spaCy (fr_core_news_md)**
- suppression des stop words
- filtrage des tokens courts (moins de 3 caractères)

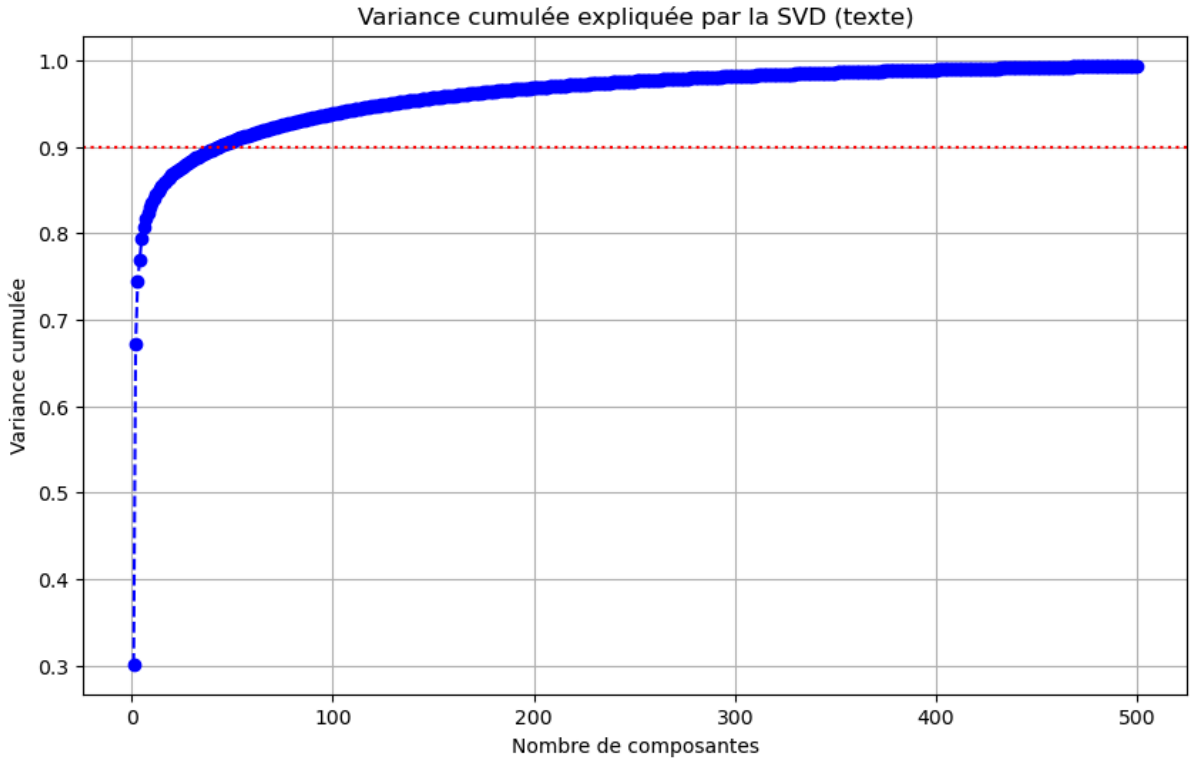
Ensuite, nous avons concaténé les variables *designation* et *description* pour former une variable textuelle unique.

Impact :

- réduction significative du bruit
- amélioration de la qualité des représentations textuelles
- homogénéisation linguistique

4.2 Vectorisation TF-IDF et réduction de dimension

Après la vectorisation TF-IDF et l'application du Truncated SVD, nous créons une **courbe de variance cumulée** pour le texte.
Cette visualisation permet de justifier le choix des 80 dimensions pour la description et 30 dimensions pour la *designation*, montrant que nous ne réduisons pas arbitrairement la dimension mais conservons la majorité de la variance informative.



Une première représentation du texte est obtenue via TF-IDF.

Paramétrage

Deux vectorisations distinctes sont réalisées :

- **description** : 7 000 features, n-grammes (1,2)
- **designation** : 2 000 features, n-grammes (1,2)

Nous avons mis en place un traitement par chunks afin de gérer efficacement la mémoire sur un dataset volumineux.

Réduction de dimension (SVD)

Les matrices TF-IDF, très creuses et de grande dimension, sont projetées dans un espace réduit via **Truncated SVD** :

- description → 80 dimensions
- designation → 30 dimensions

Les deux représentations sont ensuite concaténées puis normalisées.

Résultat :

- **110 dimensions textuelles compactes**
- conservation d'une part significative de la variance

Impact :

- réduction du coût computationnel
- limitation du risque de surapprentissage
- meilleure compatibilité avec les modèles classiques

4.3 Embeddings sémantiques avec CamemBERT

En complément du TF-IDF, une représentation sémantique avancée est extraite à l'aide du modèle **CamemBERT**.

Méthodologie

- tokenisation des textes concaténés
- troncature à 256 tokens
- traitement par batch sur GPU
- extraction du vecteur **CLS**

Chaque produit est ainsi représenté par un vecteur dense de **768 dimensions.**

Les embeddings sont ensuite sauvegardés pour éviter les recalculs.

Impact :

- capture du contexte sémantique
- meilleure compréhension des relations entre mots
- complémentarité avec TF-IDF (statistique vs sémantique)

4.4 Extraction des caractéristiques images

4.4.1 Features simples (metadata)

Des caractéristiques simples sont extraites :

- largeur
- hauteur
- taille du fichier

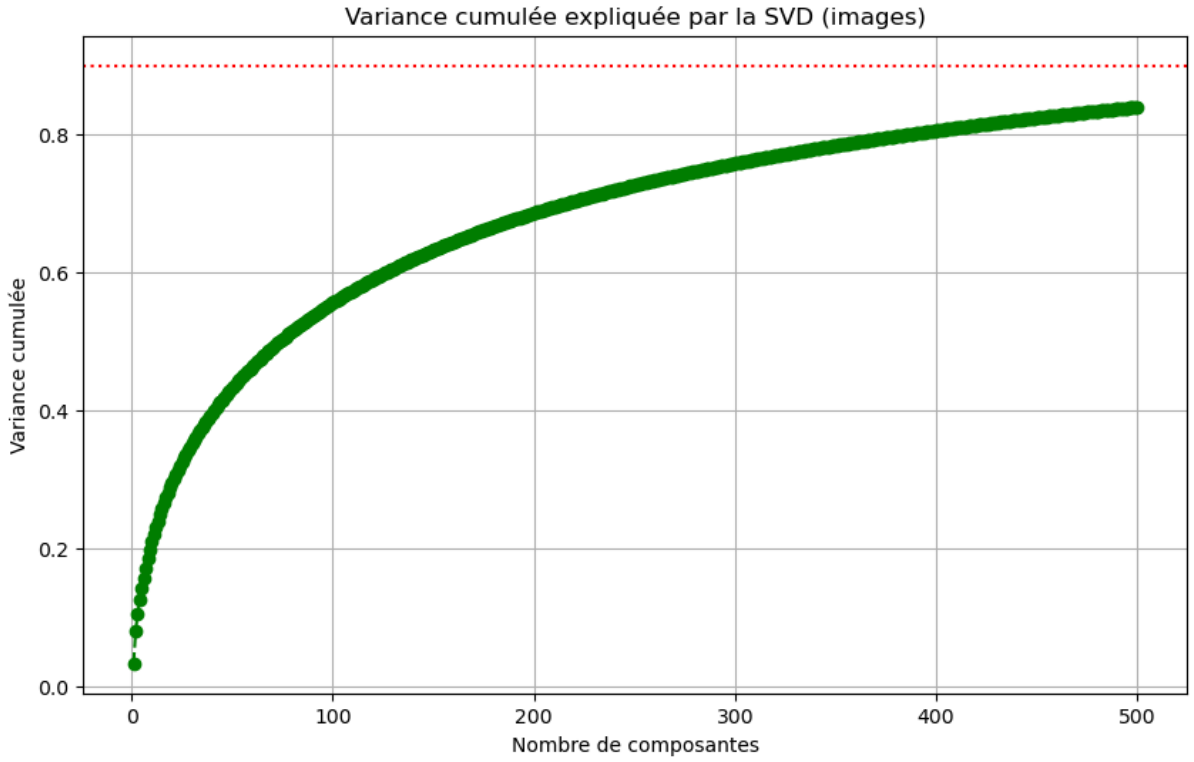
Ces variables sont ensuite normalisées entre -1 et 1.

Impact :

- apport d'une information structurelle
- faible coût computationnel

4.4.2 Embeddings visuels avec EfficientNet

Après extraction des embeddings images et application du Truncated SVD, nous créons une **courbe de variance cumulée pour les images**.
Cette visualisation permet de montrer que le choix de 250 dimensions est rationnel et capture l'essentiel de l'information visuelle, sans conserver des dimensions inutiles.



Une représentation visuelle avancée est obtenue via un modèle pré-entraîné :
EfficientNet-B3

Méthodologie

- redimensionnement des images (300×300)
- normalisation standard ImageNet
- extraction des features sans couche de classification
- traitement batché sur GPU

Chaque image est transformée en un vecteur dense.

Réduction de dimension (SVD)

Les embeddings images sont ensuite réduits via **Truncated SVD** :

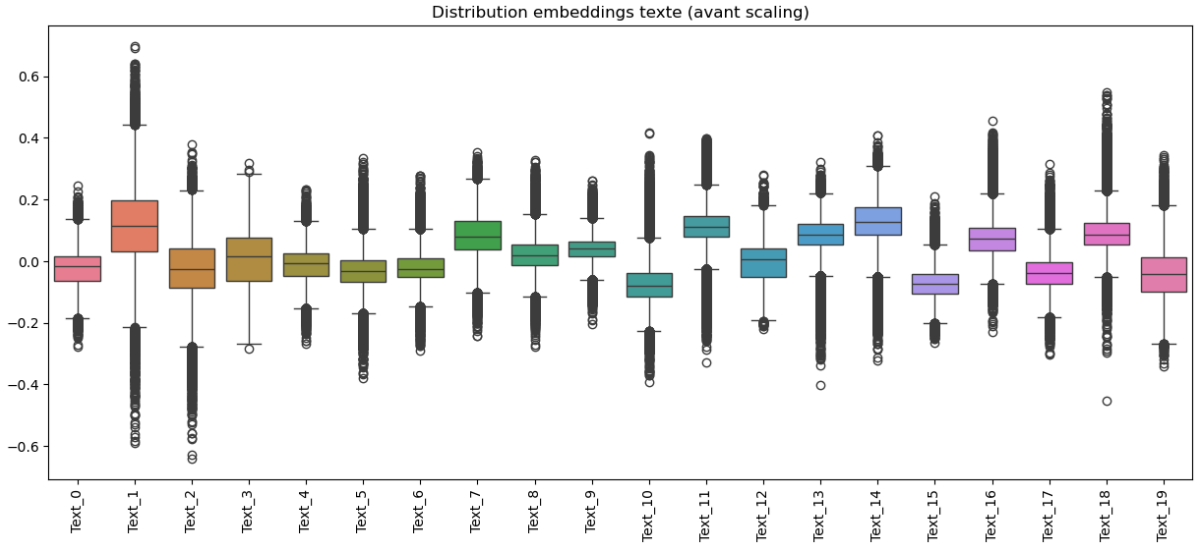
- projection à **250 dimensions**
- normalisation des features.

Impact :

- réduction du bruit
- amélioration de la robustesse
- réduction du coût mémoire

Enfin, nous créons une **visualisation de type histogramme ou boxplot** pour montrer la distribution des embeddings texte et image avant et après normalisation.

Cette étape permet de vérifier et démontrer la maîtrise des pratiques de preprocessing ML classique, utile pour l'entraînement de modèles linéaires ou de réseaux de neurones.



4.5 Fusion multimodale des features

L'ensemble des features est concaténé pour former une représentation finale unique.

Composition du vecteur final

- CamemBERT : 768 dimensions
- TF-IDF + SVD : 110 dimensions
- Images (EfficientNet + SVD) : 250 dimensions
- Metadata images : 3 dimensions

Total : 1131 dimensions

4.6 Construction du dataset final

Le dataset final est structuré comme suit :

- **X_final** : matrice de features (84 916 × 1131)
- **y_final** : variable cible (27 classes)

Vérifications

- aucune valeur manquante
- aucune valeur infinie
- distribution des classes conservée

Un split **train / validation stratifié (80/20)** est ensuite effectué afin de préserver la distribution des classes.

4.7 Optimisation et gestion mémoire

Plusieurs optimisations ont été mises en place :

- traitement par chunks (TF-IDF)
- utilisation du GPU (CamemBERT, EfficientNet)
- cache des matrices intermédiaires
- utilisation de formats optimisés (np.float32, sparse matrices)
- memmap pour les embeddings images

Impact :

- réduction du temps de calcul
- stabilité du pipeline sur environnement contraint (Kaggle)

4.8 Synthèse du preprocessing

Le pipeline de preprocessing permet de :

- transformer des données brutes en features exploitables
- combiner des approches statistiques et sémantiques
- intégrer efficacement texte et image
- produire une représentation compacte et riche

Cette étape constitue un élément central du projet, en permettant de construire un dataset :

- **riche en information**
- **robuste**
- **adapté à différents types de modèles**

5. Modèles et méthodologie expérimentale

5.1 Choix des modèles et hypothèses

Le choix des modèles suit une logique **expérimentale progressive**, visant à évaluer l'apport de chaque niveau de sophistication sur des données multimodales représentées sous forme d'embeddings :

1. **Modèles simples et interprétables** : tester si les embeddings permettent une séparation linéaire des classes.
2. **Modèles tabulaires non linéaires** : capturer les interactions complexes entre les embeddings texte, image et métadonnées.
3. **Modèles de deep learning** : exploiter pleinement la richesse des relations non linéaires et la multimodalité.
4. **Modèles combinés (stacking)** : évaluer la complémentarité des modèles précédents pour améliorer la performance globale.

Chaque modèle permet de tester une **hypothèse précise** quant à la structure de l'information contenue dans les embeddings et son exploitabilité pour la prédiction des codes produits Rakuten.

5.2 Description détaillée des modèles

5.2.1. Régression Logistique

Hypothèse testée : les embeddings permettent une séparation linéaire entre classes.

Détails techniques :

- Régression logistique multinomiale avec régularisation L2 (C = 1.0).
- Solver : lbfgs, adapté pour les problèmes multiclasses.
- Prétraitement : normalisation StandardScaler des embeddings concaténés.

Résultats :

- Accuracy : 0.7734
- F1-score macro : 0.7511

Analyse : la régression logistique confirme que les embeddings sont globalement discriminants, mais elle est limitée pour capturer des relations non linéaires.

5.2.2. LightGBM

Hypothèse testée : les interactions non linéaires entre embeddings contiennent de l'information exploitable par un modèle de gradient boosting.

Détails techniques :

- Arbres décisionnels en gradient boosting (n_estimators = 1000, learning_rate = 0.05, max_depth = 12, num_leaves = 64).
- Subsampling : 0.8, colsample_bytree : 0.8.
- Early stopping sur validation set (50 itérations).

Résultats :

- Accuracy : 0.8122
- F1-score macro : 0.7890
- LogLoss : 0.6367

Analyse : LightGBM capture efficacement les interactions texte-image et améliore significativement les performances par rapport au modèle linéaire.

5.2.3. CatBoost

Hypothèse testée : un boosting robuste aux distributions hétérogènes peut surpasser LightGBM.

Détails techniques :

- Gradient boosting sur arbres (iterations = 500, depth = 8, learning_rate = 0.1, l2_leaf_reg = 3).
- Prétraitement minimal grâce à la gestion automatique des catégories.

Résultats :

- Accuracy : 0.7453
- F1-score macro : 0.7292

Analyse : CatBoost est moins performant sur les embeddings continus que LightGBM, suggérant que sa force sur des features catégorielles n'est pas pleinement exploitée ici.

5.2.4. Multi-Layer Perceptron (MLP)

Hypothèse testée : un réseau de neurones peut capturer les relations complexes et non linéaires entre embeddings multimodaux.

Détails techniques :

- Architecture : 1141 features en entrée, couches cachées [1024, 512, 256], ReLU, Dropout 0.3.
- Optimiseur : Adam (learning_rate = 0.001), batch size 128, epochs 50, early stopping patience 5.
- Loss : CrossEntropy multiclasses.

Résultats :

- Accuracy : 0.8307
- F1-score macro : 0.8169

Analyse : le MLP exploite pleinement la complexité des embeddings et fournit la meilleure performance parmi les modèles individuels.

5.2.5. Stacking

Hypothèse testée : combiner plusieurs modèles complémentaires améliore la performance globale.

Détails techniques :

- Modèles de base : LightGBM, CatBoost, MLP
- Méta-modèle : régression logistique sur les probabilités prédites par les modèles de base
- Validation : k-fold stratifié (k = 5) pour éviter le surapprentissage

Résultats :

- Accuracy : 0.8310
- F1-score macro : 0.8166

Analyse : le stacking offre un gain marginal par rapport au MLP, ce qui indique que le MLP capture déjà la majorité de l'information discriminante.

5.3 Synthèse comparative

Modèle	Hypothèse testée	Accuracy	F1 Macro	Commentaire
Régression Logistique	Séparation linéaire possible	0.7734	0.7511	Simple et interprétable
LightGBM	Interactions non linéaires exploitables	0.8122	0.789	Meilleure capture des relations multimodales
CatBoost	Boosting robuste améliore la performance	0.7453	0.7292	Moins adapté aux embeddings continus
MLP	Capturer relations complexes	0.8307	0.8169	Meilleur modèle individuel
Stacking	Combinaison complémentaire	0.831	0.8166	Gain marginal vs MLP

Conclusion : l'approche progressive permet de démontrer l'intérêt de la complexité des modèles. Les modèles de boosting non linéaires améliorent nettement les performances par rapport au linéaire, et le MLP exploite pleinement les embeddings multimodaux. Le stacking confirme que la complémentarité existe mais apporte un bénéfice limité dans ce cas.

6. Interprétabilité

Une analyse des coefficients du meta-modèle montre :

- Contribution dominante du **MLP**
- Contributions secondaires :
    - Logistic Regression
    - LightGBM
    - CatBoost

Cela indique que :

- les embeddings + deep learning capturent mieux la structure des données
- les modèles classiques apportent peu d'information supplémentaire

7. Conclusion scientifique

Cette étude met en évidence plusieurs résultats clés :

1. Importance des embeddings

Les embeddings issus de modèles profonds permettent :

- une représentation dense et sémantique des données
- une amélioration significative des performances

2. Supériorité des modèles non linéaires

Les modèles capables de capturer des interactions complexes (MLP, boosting) surpassent :

- les modèles linéaires classiques

3. Limite du stacking

Le stacking n’apporte qu’un gain marginal lorsque :

- un modèle dominant (ici MLP) capture déjà l’essentiel de l’information

4. Impact du déséquilibre des classes

L’utilisation du **F1 macro** est essentielle pour :

- éviter un biais vers les classes majoritaires
- évaluer correctement les performances globales

8. Conclusion métier

Contexte

Le projet vise la **classification automatique de produits e-commerce (Rakuten)**.

Apports business

Le modèle permet :

- l'automatisation de la catégorisation => la réduction du travail manuel
- l'amélioration de la qualité des données => meilleure recherche produit et recommandation
- la scalabilité => traitement de volumes importants (≈85k produits)

Modèle retenu

Nous avons retenu le modèle **MLP** car :

- meilleures performances globales
- simplicité de déploiement
- temps d'inférence rapide

Cas d'usage SaaS

Le système peut être intégré dans :

- pipeline d'ingestion produit
- API de classification temps réel
- outil d'aide à la saisie vendeur

9. Perspectives futures

9.1 Améliorations possibles

1. **Data – enrichissement des images**
    - Perspective : intégrer des images de meilleure qualité ou plus nombreuses pour améliorer la représentation visuelle des produits.
    - Pourquoi non fait : le projet s'est limité aux images disponibles dans le jeu de données pour des contraintes de temps et de disponibilité des données.
2. **Data – augmentation des données**
    - Perspective : générer des variations d'images ou de textes pour accroître la diversité du jeu d'entraînement.
    - Pourquoi non fait : seules des techniques simples ont été appliquées afin de valider le pipeline de base ; les augmentations avancées demanderaient un temps de calcul et de validation supplémentaires.
3. **Modèles – Transformers multimodaux (CLIP, BLIP)**
    - Perspective : fine-tuning de modèles multimodaux end-to-end pour mieux exploiter la correspondance texte-image.
    - Pourquoi non fait : ces modèles nécessitent des ressources GPU importantes et un temps d'entraînement long, ce qui dépassait le cadre temporel et matériel du projet.
4. **Modèles – fine-tuning CamemBERT**
    - Perspective : adapter CamemBERT aux données spécifiques du catalogue Rakuten pour améliorer les embeddings texte.
    - Pourquoi non fait : le fine-tuning complet aurait nécessité un temps d'expérimentation supplémentaire et des ressources computationnelles non disponibles.
5. **Industrialisation – monitoring du drift et retraining automatique**
    - Perspective : mettre en place un suivi automatique de la performance et des mises à jour périodiques du modèle.
    - Pourquoi non fait : le projet a été centré sur la validation scientifique et la démonstration de faisabilité ; l'industrialisation relève d'une phase ultérieure de production.
6. **Industrialisation – A/B testing**
    - Perspective : tester différentes versions du modèle en production pour identifier la plus performante en conditions réelles.
    - Pourquoi non fait : cette étape nécessite un environnement de production et des utilisateurs finaux, ce qui dépasse le périmètre du projet académique/expérimental.
    -